



**Politecnico
di Torino**



Advanced Machine Learning

A.A. 2023/2024

Tatiana Tommasi

The Perceptron

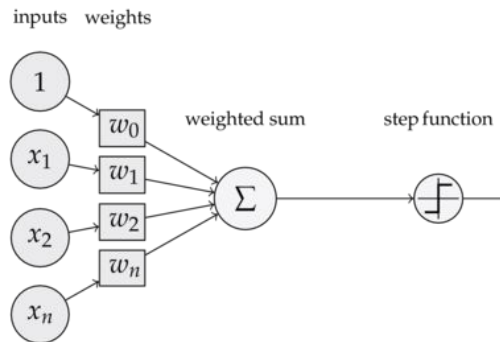


Frank Rosenblatt
1928–1969

Rosenblatt's perceptron played an important role in the history of machine learning. Initially, Rosenblatt simulated the perceptron on an IBM 704 computer at Cornell in 1957, but by the early 1960s he had built

special-purpose hardware that provided a direct, parallel implementation of perceptron learning. Many of his ideas were encapsulated in "Principles of Neurodynamics: Perceptrons and the Theory of Brain Mechanisms" published in 1962. Rosenblatt's work was criticized by Marvin Minsky, whose objections were published in the book "Perceptrons", co-authored with

Seymour Papert. This book was widely misinterpreted at the time as showing that neural networks were fatally flawed and could only learn solutions for linearly separable problems. In fact, it only proved such limitations in the case of single-layer networks such as the perceptron and merely conjectured (incorrectly) that they applied to more general network models. Unfortunately, however, this book contributed to the substantial decline in research funding for neural computing, a situation that was not reversed until the mid-1980s. Today, there are many hundreds, if not thousands, of applications of neural networks in widespread use, with examples in areas such as handwriting recognition and information retrieval being used routinely by millions of people.



Figures from the book "Pattern Recognition and Machine Learning", Bishop

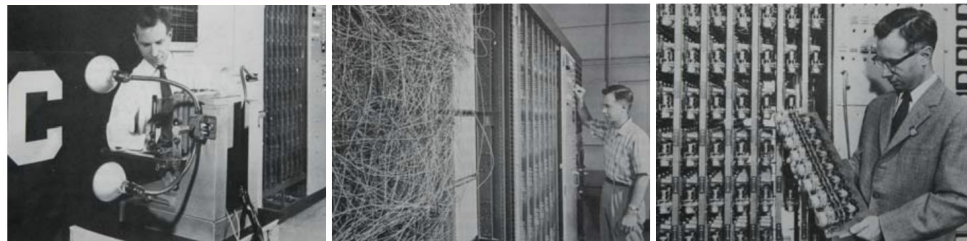
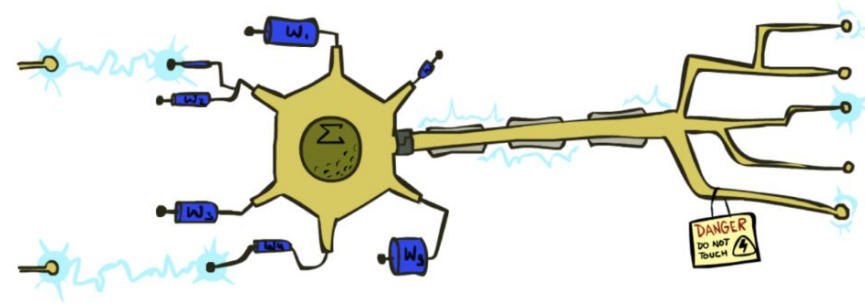
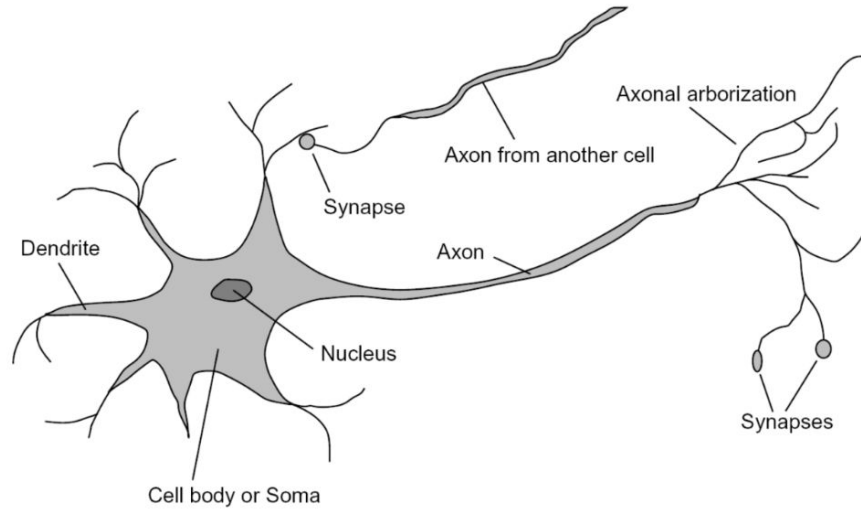


Figure 4.8 Illustration of the Mark 1 perceptron hardware. The photograph on the left shows how the inputs were obtained using a simple camera system in which an input scene, in this case a printed character, was illuminated by powerful lights, and an image focussed onto a 20×20 array of cadmium sulphide photocells, giving a primitive 400 pixel image. The perceptron also had a patch board, shown in the middle photograph, which allowed different configurations of input features to be tried. Often these were wired up at random to demonstrate the ability of the perceptron to learn without the need for precise wiring, in contrast to a modern digital computer. The photograph on the right shows one of the racks of adaptive weights. Each weight was implemented using a rotary variable resistor, also called a potentiometer, driven by an electric motor thereby allowing the value of the weight to be adjusted automatically by the learning algorithm.

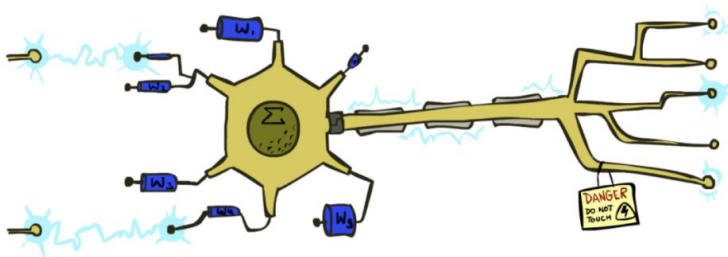
The Perceptron

- Very loose inspiration: human neurons



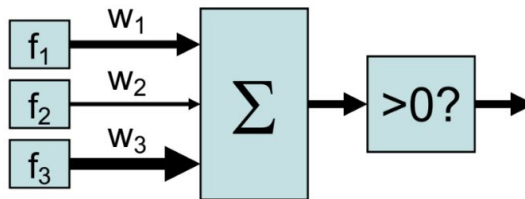
The Perceptron: Linear Classifier

- Inputs are **feature values**
- Each feature has a **weight**
- Sum is the **activation**

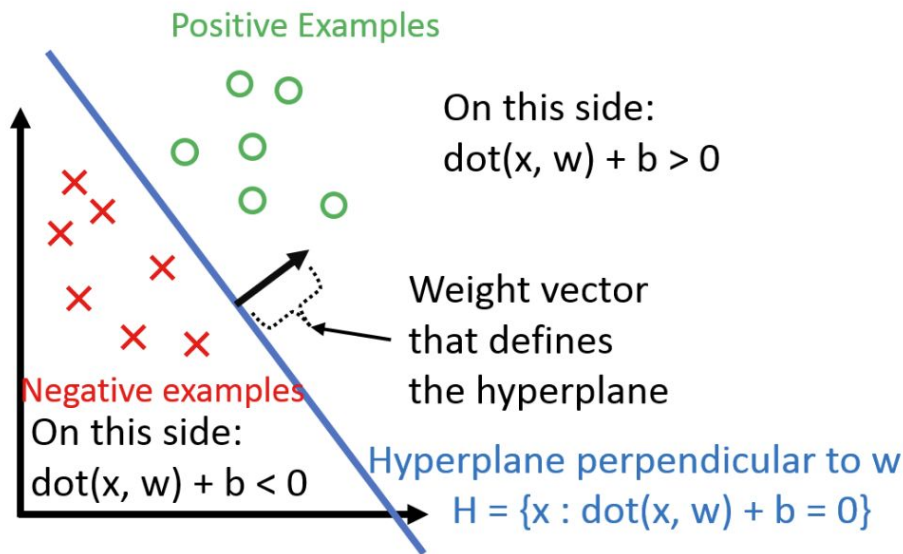


$$\text{activation}_w(x) = \sum_i w_i \cdot x = w \cdot x$$

- If the activation is:
 - Positive, output +1
 - Negative, output -1



The Perceptron: Linear Classifier



$$h(x_i) = \text{sign}(\mathbf{w}^\top \mathbf{x}_i + b)$$

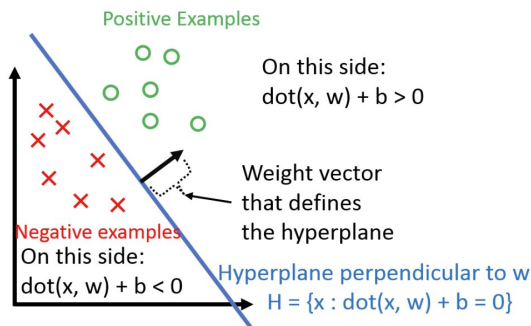
$$\mathbf{x}_i \text{ becomes } \begin{bmatrix} \mathbf{x}_i \\ 1 \end{bmatrix}$$

$$\mathbf{w} \text{ becomes } \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix}$$

$$\begin{bmatrix} \mathbf{x}_i \\ 1 \end{bmatrix}^\top \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix} = \mathbf{w}^\top \mathbf{x}_i + b$$

$$h(\mathbf{x}_i) = \text{sign}(\mathbf{w}^\top \mathbf{x})$$

The Perceptron: Linear Classifier



$$h(\mathbf{x}_i) = \text{sign}(\mathbf{w}^\top \mathbf{x})$$

Observation: Note that

$$y_i(\mathbf{w}^\top \mathbf{x}_i) > 0 \iff \mathbf{x}_i \text{ is classified correctly}$$

where 'classified correctly' means that $\mathbf{x}_i$ is on the correct side of the hyperplane defined by $\mathbf{w}$. Also, note that the left side depends on $y_i \in \{-1, +1\}$ (it wouldn't work if, for example $y_i \in \{0, +1\}$).

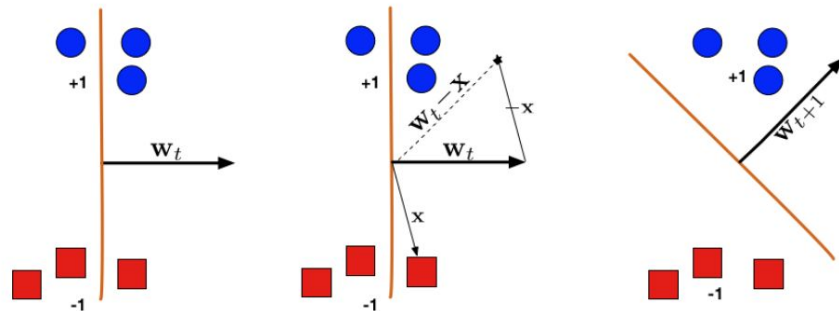
The Perceptron: Linear Classifier

- Start with weights = 0
- For each training instance:
 - Classify with current weights

$$y = \begin{cases} +1 & \text{if } w \cdot x \geq 0 \\ -1 & \text{if } w \cdot x < 0 \end{cases}$$

- If correct (i.e., $y=y^*$), no change!
- If wrong: adjust the weight vector by adding or subtracting the feature vector. Subtract if y^* is -1.

$$w = w + y^* \cdot x$$

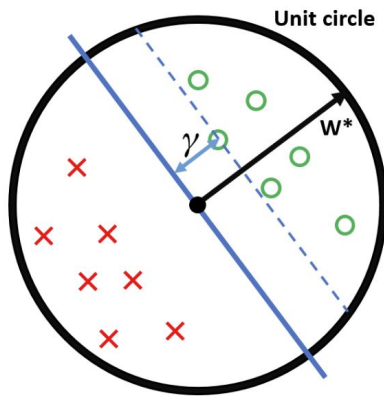


Perceptron Convergence

The Perceptron was arguably the first algorithm with **a strong formal guarantee**.

If a data set is linearly separable, the Perceptron will find a separating hyperplane in a finite number of updates

If the data is not linearly separable, it will loop forever!

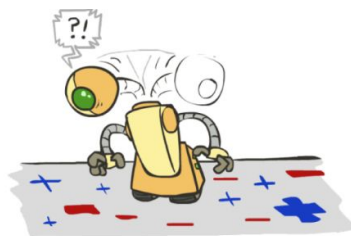
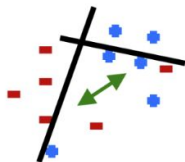


- All inputs $\mathbf{x}_i$ live within the unit sphere
- There exists a separating hyperplane defined by $\mathbf{w}^*$, with $\|\mathbf{w}\|^* = 1$ (i.e. $\mathbf{w}^*$ lies exactly on the unit sphere).
- γ is the distance from this hyperplane (blue) to the closest data point.

Theorem: If all of the above holds, then the Perceptron algorithm makes at most $1/\gamma^2$ mistakes.

Problems with the Perceptron

- Noise: if the data isn't separable, weights might thrash



Perceptron Algorithm

```
Initialize  $\vec{w} = \vec{0}$ 
while TRUE do
   $m = 0$ 
  for  $(x_i, y_i) \in D$  do
    if  $y_i(\vec{w}^T \cdot \vec{x}_i) \leq 0$  then
       $\vec{w} \leftarrow \vec{w} + y_i \vec{x}_i$ 
       $m \leftarrow m + 1$ 
    end if
  end for
  if  $m = 0$  then
    break
  end if
end while
```

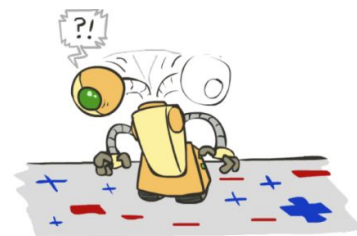
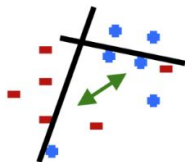
```
// Initialize  $\vec{w}$ .  $\vec{w} = \vec{0}$  misclassifies everything.
// Keep looping
// Count the number of misclassifications,  $m$ 
// Loop over each (data, label) pair in the dataset,  $D$ 
// If the pair  $(\vec{x}_i, y_i)$  is misclassified
// Update the weight vector  $\vec{w}$ 
// Counter the number of misclassification

// If the most recent  $\vec{w}$  gave 0 misclassifications
// Break out of the while-loop

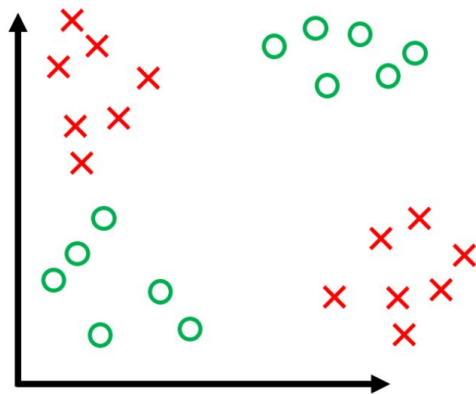
// Otherwise, keep looping!
```

Problems with the Perceptron

- Noise: if the data isn't separable, weights might thrash



- Initially, huge wave of excitement ("Digital brains") (See [The New Yorker December 1958](#))
- Then, contributed to the A.I. Winter. Famous example of a simple non-linearly separable data set, the XOR problem (Minsky 1969):



A	B	A AND B	A OR B	NOT A
False	False	False	False	True
False	True	False	True	True
True	False	False	True	False
True	True	True	True	False

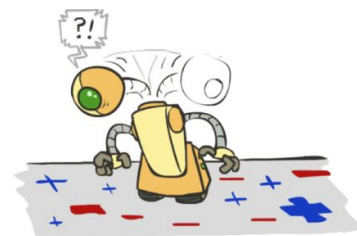
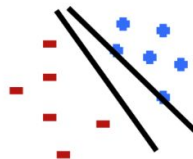
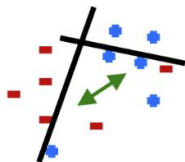
A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

$$XOR(x1, x2) = \text{AND}(\text{NOT}(\text{AND}(x1, x2)), \text{OR}(x1, x2))$$

<https://towardsdatascience.com/perceptrons-logical-functions-and-the-xor-problem-37ca5025790a>

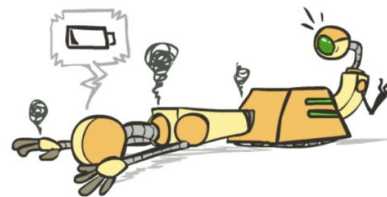
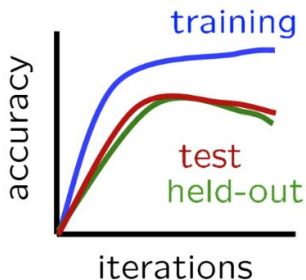
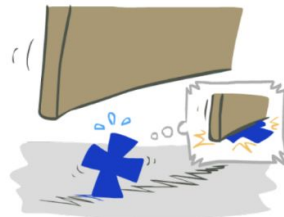
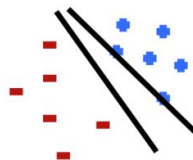
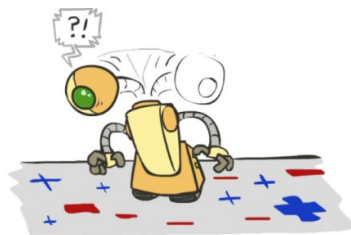
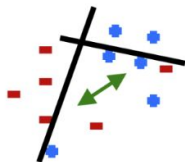
Problems with the Perceptron

- Noise: if the data isn't separable, weights might thrash
- Mediocre generalization: finds a "barely" separating solution



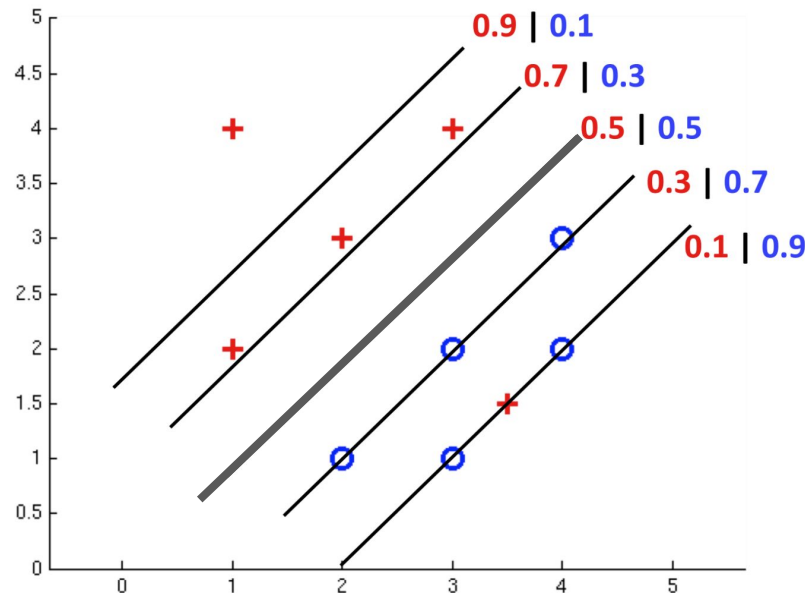
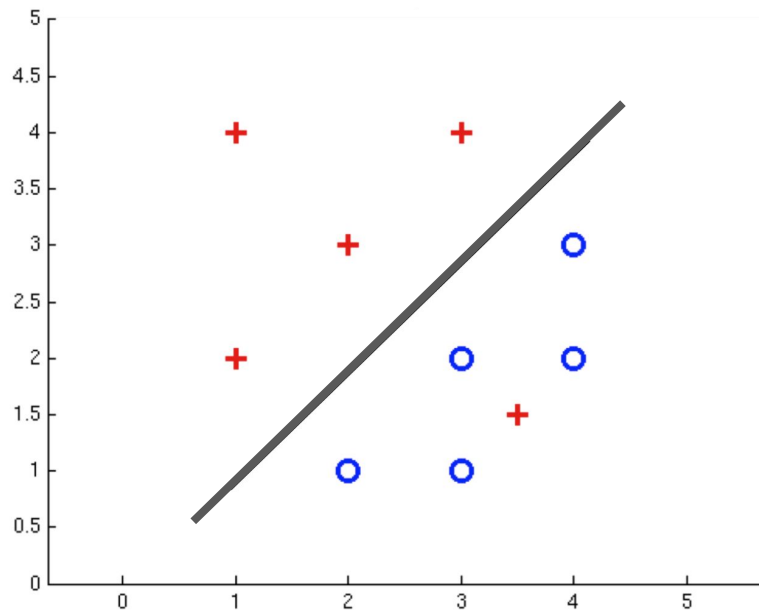
Problems with the Perceptron

- Noise: if the data isn't separable, weights might thrash
- Mediocre generalization: finds a "barely" separating solution
- Overtraining: test / held-out accuracy usually rises, then falls
 - Overtraining is a kind of overfitting



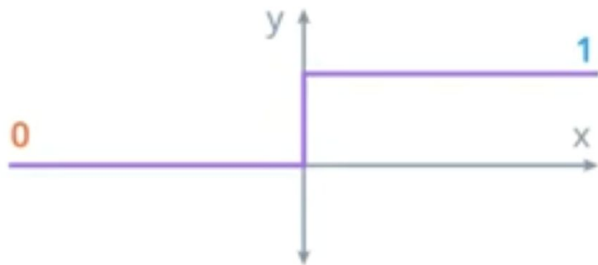
Improving the Perceptron

From Deterministic to Probabilistic decision



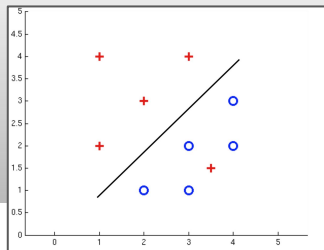
Improving the Perceptron

From Deterministic to Probabilistic decision



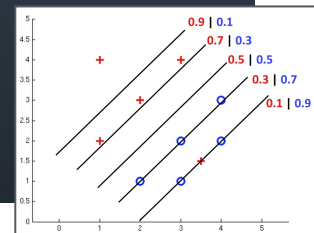
DISCRETE:
Step Function

$$y = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$$

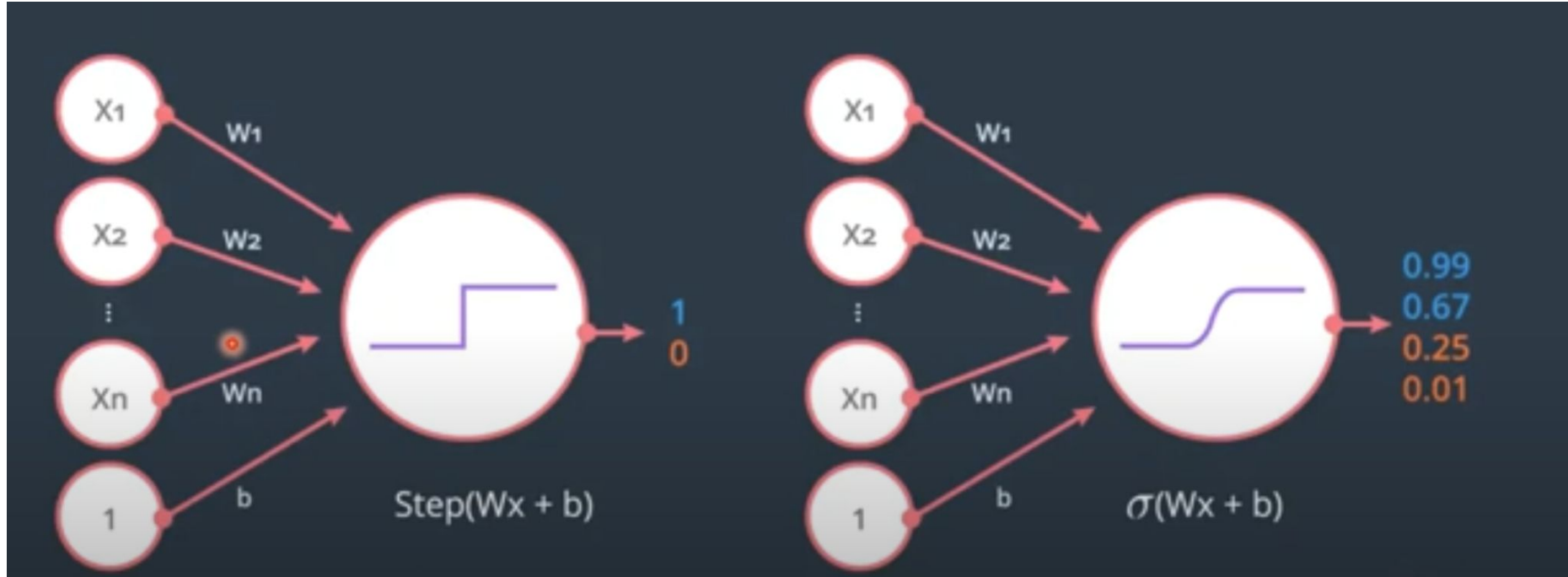


CONTINUOUS:
Sigmoid Function

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



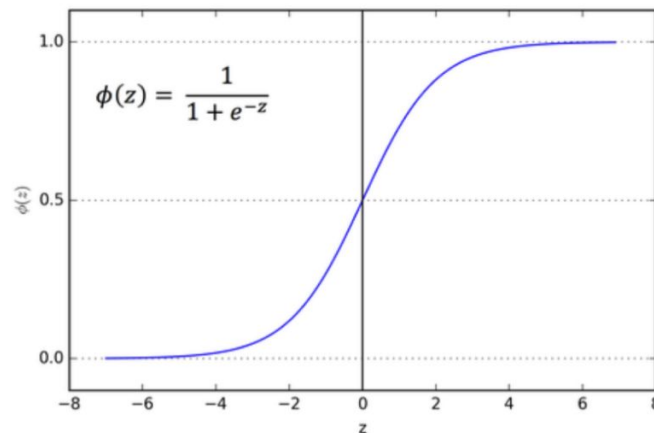
Improving the Perceptron



Logistic Regression

- Perceptron scoring: $z = w \cdot x$
- If $z = w \cdot x$ very positive $\rightarrow$ want probability going to 1
- If $z = w \cdot x$ very negative $\rightarrow$ want probability going to 0
- Sigmoid function

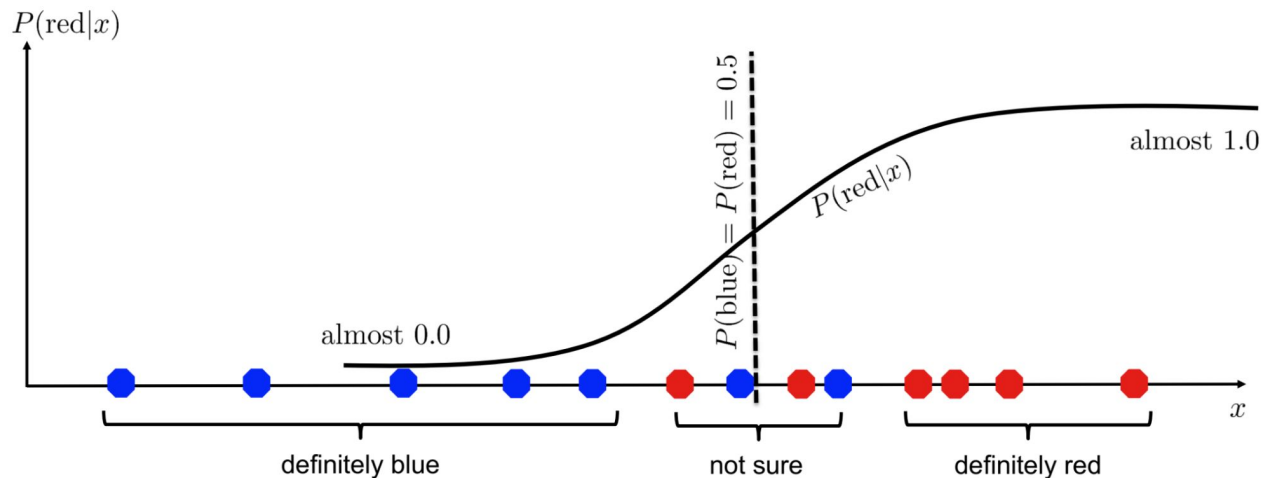
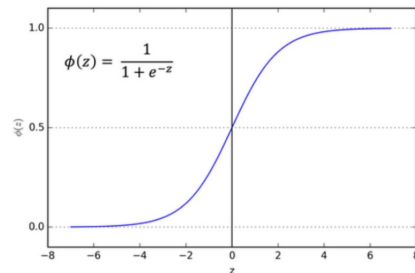
$$\phi(z) = \frac{1}{1 + e^{-z}}$$



Logistic Regression

- Sigmoid function

$$\phi(z) = \frac{1}{1 + e^{-z}}$$



$$P(\text{red}|x) = \frac{e^{w_{\text{red}} \cdot x}}{e^{w_{\text{red}} \cdot x} + e^{w_{\text{blue}} \cdot x}}$$

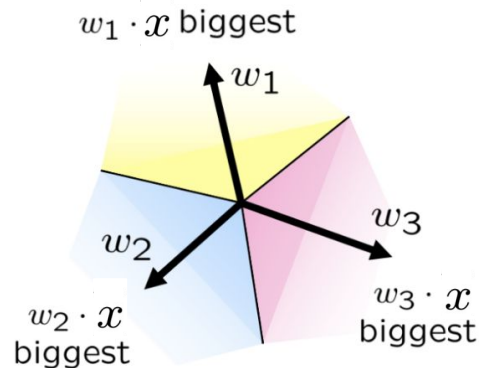
probability increases exponentially as we move away from boundary

normalizer

Multiclass Logistic Regression

- Recall Perceptron:

- A weight vector for each class: w_y
- Score (activation) of a class y : $w_y \cdot x$
- Prediction highest score wins $y = \arg \max_y w_y \cdot x$



- How to make the scores into probabilities?

$$\underbrace{z_1, z_2, z_3}_{\text{original activations}} \rightarrow \underbrace{\frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}}, \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3}}, \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3}}}_{\text{softmax activations}}$$

How to find the best w ?

- Maximum likelihood estimation:

$$\max_w ll(w) = \max_w \sum_i \log P(y^{(i)}|x^{(i)}; w)$$

with:

$$P(y^{(i)} = +1|x^{(i)}; w) = \frac{1}{1 + e^{-w \cdot f(x^{(i)})}}$$
$$P(y^{(i)} = -1|x^{(i)}; w) = 1 - \frac{1}{1 + e^{-w \cdot f(x^{(i)})}}$$

= Logistic Regression

- Maximum likelihood estimation:

$$\max_w ll(w) = \max_w \sum_i \log P(y^{(i)}|x^{(i)}; w)$$

with:

$$P(y^{(i)}|x^{(i)}; w) = \frac{e^{w_{y^{(i)}} \cdot f(x^{(i)})}}{\sum_y e^{w_y \cdot f(x^{(i)})}}$$

= Multi-Class Logistic Regression

SGD and Perceptron

- Hypothesis: $y = \text{sign}(w^T x)$
- Define hinge loss

$$l(w, x_t, y_t) = -y_t w^T x_t \mathbb{I}[\text{mistake on } x_t]$$

$$\hat{L}(w) = - \sum_t y_t w^T x_t \mathbb{I}[\text{mistake on } x_t]$$

$$w_{t+1} = w_t - \eta_t \nabla l(w_t, x_t, y_t) = w_t + \eta_t y_t x_t \mathbb{I}[\text{mistake on } x_t]$$

- Set $\eta_t = 1$. If mistake on a positive example

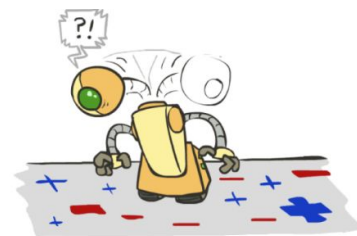
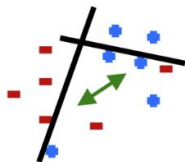
$$w_{t+1} = w_t + y_t x_t = w_t + x$$

- If mistake on a negative example

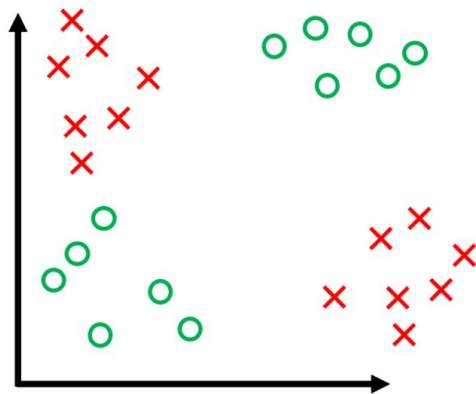
$$w_{t+1} = w_t + y_t x_t = w_t - x$$

Problems with the Perceptron

- Noise: if the data isn't separable, weights might thrash



- Initially, huge wave of excitement ("Digital brains") (See [The New Yorker December 1958](#))
- Then, contributed to the A.I. Winter. Famous example of a simple non-linearly separable data set, the XOR problem (Minsky 1969):



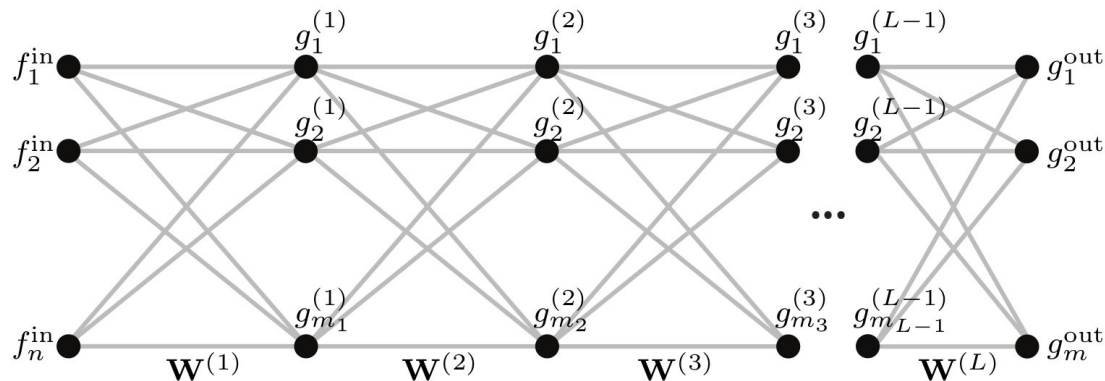
A	B	A AND B	A OR B	NOT A
False	False	False	False	True
False	True	False	True	True
True	False	False	True	False
True	True	True	True	False

A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

$$XOR(x1, x2) = \text{AND}(\text{NOT}(\text{AND}(x1, x2)), \text{OR}(x1, x2))$$

<https://towardsdatascience.com/perceptrons-logical-functions-and-the-xor-problem-37ca5025790a>

Neural Network



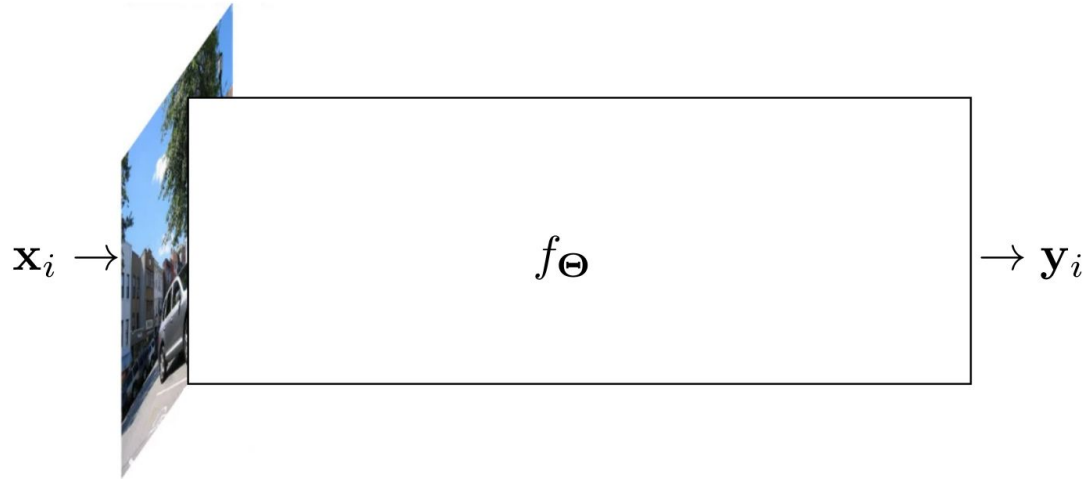
Deep neural network consisting of L layers

Linear layer $\mathbf{g}^{(k)} = \sigma(\mathbf{W}^{(k)} \mathbf{g}^{(k-1)})$

Activation, e.g. $\sigma(x) = \max\{x, 0\}$ rectified linear unit (ReLU)

Parameters weights of all layers $\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(L)}$ (including biases)

Advanced Models: Deep Learning



Advanced Models

In practice, many interesting phenomena are **highly nonlinear**.

We must choose a good learning model to capture these phenomena.

Advanced Models

In practice, many interesting phenomena are **highly nonlinear**.

We must choose a good learning model to capture these phenomena.

A powerful model should be as **universal** as possible.

The general recipe is always the following:

- **Fix** the general form for the parametric model.
- **Optimize** for the parameters.

Bonus:

- The model should be easy to work with.

Deep Composition

The simplest example of a nonlinear parametric model:

$$f \circ f(\mathbf{x})$$

Deep Composition

The simplest example of a nonlinear parametric model:

$$\underbrace{f}_{\text{linear}} \circ \underbrace{f}_{\text{linear}}(\mathbf{x})$$

$$\underbrace{f \circ f}_{\text{linear}}(\mathbf{x})$$

Deep Composition

The simplest example of a nonlinear parametric model:

$$\sigma \circ f(\mathbf{x})$$

If σ is the logistic function, we have the **logistic regression** model.

Consider multiple **layers** of logistic regression models:

$$\text{output} \leftarrow \underbrace{(\sigma \circ f)}_{\text{layer } n} \circ (\sigma \circ f) \circ \cdots \circ \underbrace{(\sigma \circ f)}_{\text{layer } 1}(\mathbf{x}) \leftarrow \text{input}$$

Deep Composition

The simplest example of a nonlinear parametric model:

$$\sigma \circ f(\mathbf{x})$$

If σ is the logistic function, we have the **logistic regression** model.

Consider multiple **layers** of logistic regression models:

$$\text{output} \leftarrow \underbrace{(\sigma \circ f)}_{\text{output layer}} \circ (\sigma \circ f) \circ \cdots \circ \underbrace{(\sigma \circ f)}_{\text{input layer}}(\mathbf{x}) \leftarrow \text{input}$$

(Function composition is associative, so parentheses are not necessary.)

More in general, consider other **activation functions** than logistic:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \qquad \sigma(x) = \max\{0, x\}$$

continuous

discontinuous
gradient

Multi-Layer Perceptron

We call the composition with linear f and nonlinear σ :

$$g_{\Theta}(\mathbf{x}) = (\sigma \circ f_{\Theta_n}) \circ (\sigma \circ f_{\Theta_{n-1}}) \circ \cdots \circ (\sigma \circ f_{\Theta_1})(\mathbf{x})$$

a multi-layer perceptron (MLP) or deep feed-forward neural network.

Multi-Layer Perceptron

We call the composition with linear f and nonlinear σ :

$$g_{\Theta}(\mathbf{x}) = (\sigma \circ f_{\Theta_n}) \circ (\sigma \circ f_{\Theta_{n-1}}) \circ \cdots \circ (\sigma \circ f_{\Theta_1})(\mathbf{x})$$

a multi-layer perceptron (MLP) or deep feed-forward neural network.

The parameters or weights of the MLP are scattered across the layers.

Multi-Layer Perceptron

We call the composition with linear f and nonlinear σ :

$$g_{\Theta}(\mathbf{x}) = (\sigma \circ f_{\Theta_n}) \circ (\sigma \circ f_{\Theta_{n-1}}) \circ \cdots \circ (\sigma \circ f_{\Theta_1})(\mathbf{x})$$

a **multi-layer perceptron** (MLP) or **deep feed-forward neural network**.

The parameters or **weights** of the MLP are scattered across the layers.

Each layer outputs an intermediate **hidden representation**:

$$\mathbf{x}_{\ell+1} = \sigma_{\ell}(\mathbf{W}_{\ell}\mathbf{x}_{\ell} + \mathbf{b}_{\ell})$$

where we encode the weights at layer ℓ in the matrix $\mathbf{W}_{\ell}$ and bias $\mathbf{b}_{\ell}$.

Multi-Layer Perceptron

We call the composition with linear f and nonlinear σ :

$$g_{\Theta}(\mathbf{x}) = (\sigma \circ f_{\Theta_n}) \circ (\sigma \circ f_{\Theta_{n-1}}) \circ \cdots \circ (\sigma \circ f_{\Theta_1})(\mathbf{x})$$

a **multi-layer perceptron** (MLP) or **deep feed-forward neural network**.

The parameters or **weights** of the MLP are scattered across the layers.

Each layer outputs an intermediate **hidden representation**:

$$\mathbf{x}_{\ell+1} = \sigma_{\ell}(\mathbf{W}_{\ell}\mathbf{x}_{\ell} + \mathbf{b}_{\ell})$$

where we encode the weights at layer ℓ in the matrix $\mathbf{W}_{\ell}$ and bias $\mathbf{b}_{\ell}$.

Remark: The **bias** can be integrated inside the weight matrix by writing:

$$\mathbf{W} \mapsto \begin{pmatrix} \mathbf{W} & \mathbf{b} \end{pmatrix}, \quad \mathbf{x} \mapsto \begin{pmatrix} \mathbf{x} \\ 1 \end{pmatrix},$$

because each f is **linear in the parameters** just like in linear regression.

Hidden Units

At each **hidden layer** we have:

$$\mathbf{x}_{\ell+1} = \sigma_{\ell}(\mathbf{W}_{\ell}\mathbf{x}_{\ell})$$

Hidden Units

At each **hidden layer** we have:

$$\mathbf{x}_{\ell+1} = \sigma_{\ell}(\mathbf{W}_{\ell}\mathbf{x}_{\ell})$$

Each row of the weight matrix is called a **neuron** or **hidden unit**:

$$\mathbf{W}_{\mathbf{x}} = \begin{pmatrix} \text{--- unit ---} \\ \vdots \\ \text{--- unit ---} \end{pmatrix} \begin{pmatrix} | \\ \mathbf{x} \\ | \end{pmatrix}$$

Hidden Units

At each **hidden layer** we have:

$$\mathbf{x}_{\ell+1} = \sigma_{\ell}(\mathbf{W}_{\ell}\mathbf{x}_{\ell})$$

Each row of the weight matrix is called a **neuron** or **hidden unit**:

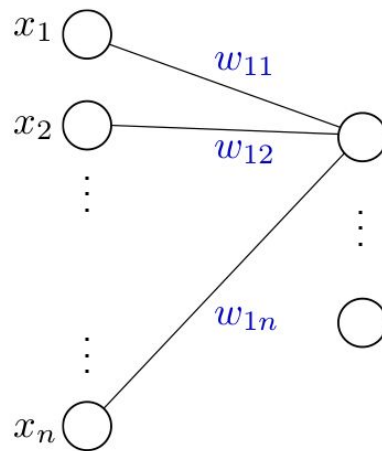
$$\mathbf{W}_{\mathbf{x}} = \begin{pmatrix} \text{--- unit ---} \\ \vdots \\ \text{--- unit ---} \end{pmatrix} \begin{pmatrix} | \\ \mathbf{x} \\ | \end{pmatrix}$$

We have two interpretations:

- ① Each layer is a vector-to-vector function $\mathbb{R}^p \rightarrow \mathbb{R}^q$.
- ② Each layer has q units acting **in parallel**.
Each unit acts as a scalar function $\mathbb{R}^p \rightarrow \mathbb{R}$.

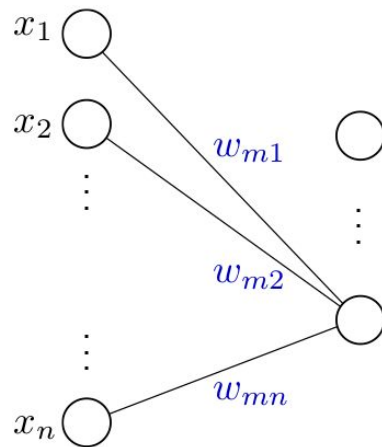
Single Layer Illustration

$$\sigma(\mathbf{W}\mathbf{x}) = \sigma \circ \begin{pmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \cdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \sigma \circ \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix}$$



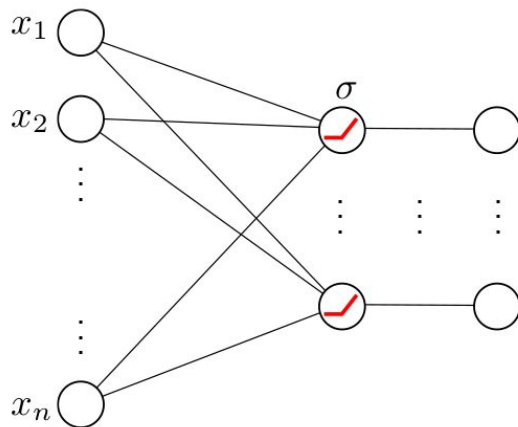
Single Layer Illustration

$$\sigma(\mathbf{W}\mathbf{x}) = \sigma \circ \begin{pmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \cdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \sigma \circ \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix}$$



Single Layer Illustration

$$\sigma(\mathbf{W}\mathbf{x}) = \sigma \circ \begin{pmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \cdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \sigma \circ \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix}$$



Output Layer

The output layer determines the co-domain of the network:

$$\mathbf{y} = (\sigma \circ f) \circ (\sigma \circ f) \circ \cdots \circ (\sigma \circ f)(\mathbf{x})$$

Output Layer

The output layer determines the co-domain of the network:

$$\mathbf{y} = (\sigma \circ f) \circ (\sigma \circ f) \circ \cdots \circ (\sigma \circ f)(\mathbf{x})$$

If σ is the logistic sigmoid, then the entire network will map:

$$\mathbb{R}^p \rightarrow (0, 1)^q$$

Output Layer

The output layer determines the co-domain of the network:

$$\mathbf{y} = (\sigma \circ f) \circ (\sigma \circ f) \circ \cdots \circ (\sigma \circ f)(\mathbf{x})$$

If σ is the logistic sigmoid, then the entire network will map:

$$\mathbb{R}^p \rightarrow (0, 1)^q$$

For generality, it is common to have a **linear** layer at the output:

$$\mathbf{y} = f \circ (\sigma \circ f) \circ \cdots \circ (\sigma \circ f)(\mathbf{x})$$

mapping:

$$\mathbb{R}^p \rightarrow \mathbb{R}^q$$

Deep ReLU networks

Adding a linear layer at the output:

$$\mathbf{y} = f \circ \sigma(\cdots)(\mathbf{x})$$

expresses $\mathbf{y}$ as a combination of “ridge functions” $\sigma(\cdots)$.

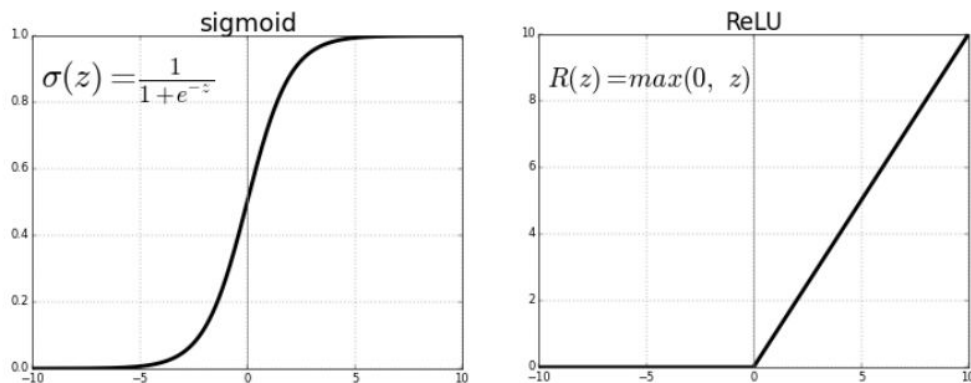
Deep ReLU networks

Adding a linear layer at the output:

$$\mathbf{y} = f \circ \sigma(\cdots)(\mathbf{x})$$

expresses $\mathbf{y}$ as a combination of “ridge functions” $\sigma(\cdots)$.

For a 2-layer network with activation $\sigma(x) = \max\{0, x\}$ (**rectifier**), we get a **piecewise-linear** function:



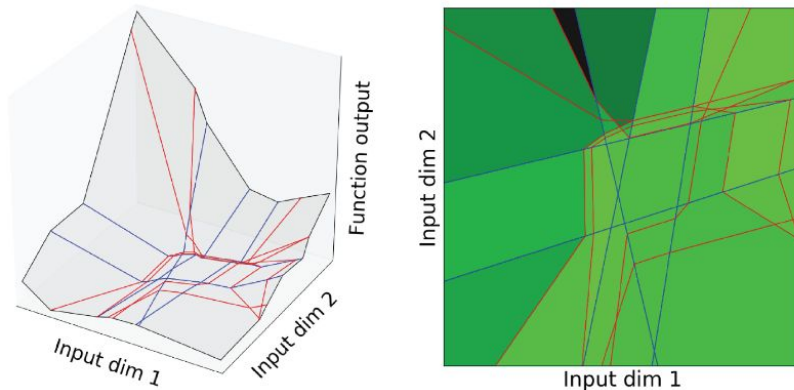
Deep ReLU networks

Adding a linear layer at the output:

$$\mathbf{y} = f \circ \sigma(\cdots)(\mathbf{x})$$

expresses $\mathbf{y}$ as a combination of “ridge functions” $\sigma(\cdots)$.

For a 2-layer network with activation $\sigma(x) = \max\{0, x\}$ (**rectifier**), we get a **piecewise-linear** function:



The blue and red edges are produced by the **first** and **second** layer.

Universality

What class of functions can we represent with a MLP?

Universality

What class of functions can we represent with a MLP?

If σ is sigmoidal, we have the following:

Universal Approximation Theorem For any compact set $\Omega \subset \mathbb{R}^p$, the space spanned by the functions $\phi(\mathbf{x}) = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$ is dense in $\mathcal{C}(\Omega)$ for the uniform convergence.

Universality

What class of functions can we represent with a MLP?

If σ is sigmoidal, we have the following:

Universal Approximation Theorem For any compact set $\Omega \subset \mathbb{R}^p$, the space spanned by the functions $\phi(\mathbf{x}) = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$ is dense in $\mathcal{C}(\Omega)$ for the uniform convergence. Thus, for any continuous function f and any $\epsilon > 0$, there exists $q \in \mathbb{N}$ and weights s.t.:

$$|f(\mathbf{x}) - \sum_{k=1}^q u_k \phi(\mathbf{x})| \leq \epsilon \quad \text{for all } \mathbf{x} \in \Omega$$

Universality

What class of functions can we represent with a MLP?

If σ is sigmoidal, we have the following:

Universal Approximation Theorem For any compact set $\Omega \subset \mathbb{R}^p$, the space spanned by the functions $\phi(\mathbf{x}) = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$ is dense in $\mathcal{C}(\Omega)$ for the uniform convergence. Thus, for any continuous function f and any $\epsilon > 0$, there exists $q \in \mathbb{N}$ and weights s.t.:

$$|f(\mathbf{x}) - \sum_{k=1}^q u_k \phi(\mathbf{x})| \leq \epsilon \quad \text{for all } \mathbf{x} \in \Omega$$

The network in the theorem has just **one** hidden layer.

For large enough q , the training error can be made **arbitrarily small**.

Cybenko, "Approximations by superpositions of sigmoidal functions", 1989

Universality

UAT theorems exist for other activations like ReLUs and locally bounded non-polynomials.

These proofs are **not constructive**.

They do not say how to compute the weights to reach a desired accuracy.

Leshno et al, "Multilayer Feedforward Networks With a Nonpolynomial Activation Function Can Approximate Any Function", 1993; Lu et al, "The Expressive Power of Neural Networks: A View from the Width", NIPS 2017

Universality

UAT theorems exist for other activations like ReLUs and locally bounded non-polynomials.

These proofs are **not constructive**.

They do not say how to compute the weights to reach a desired accuracy.

Some theorems give bounds for the **width** q (“# of neurons”).

Some theorems show universality for **> 1 layers** (deep networks).

In general, we deal with nonconvex functions. Empirical results show that large q + gradient descent leads to very good approximations.

Leshno et al, “Multilayer Feedforward Networks With a Nonpolynomial Activation Function Can Approximate Any Function”, 1993; Lu et al, “The Expressive Power of Neural Networks: A View from the Width”, NIPS 2017

Training

Given a MLP with training pairs $\{\mathbf{x}_i, \mathbf{y}_i\}$:

$$g_{\Theta}(\mathbf{x}_i) = (\sigma \circ f_{\Theta_n}) \circ (\sigma \circ f_{\Theta_{n-1}}) \circ \cdots \circ (\sigma \circ f_{\Theta_1})(\mathbf{x}_i) = \mathbf{y}_i$$

Training

Given a MLP with training pairs $\{\mathbf{x}_i, \mathbf{y}_i\}$:

$$g_{\Theta}(\mathbf{x}_i) = (\sigma \circ f_{\Theta_n}) \circ (\sigma \circ f_{\Theta_{n-1}}) \circ \cdots \circ (\sigma \circ f_{\Theta_1})(\mathbf{x}_i) = \mathbf{y}_i$$

Consider the MSE loss:

$$\ell_{\Theta}(\{\mathbf{x}_i, \mathbf{y}_i\}) = \frac{1}{n} \sum_{i=1}^n \|\mathbf{y}_i - g_{\Theta}(\mathbf{x}_i)\|_2^2$$

Solving for the weights Θ is referred to as **training**.

Training

Given a MLP with training pairs $\{\mathbf{x}_i, \mathbf{y}_i\}$:

$$g_{\Theta}(\mathbf{x}_i) = (\sigma \circ f_{\Theta_n}) \circ (\sigma \circ f_{\Theta_{n-1}}) \circ \dots \circ (\sigma \circ f_{\Theta_1})(\mathbf{x}_i) = \mathbf{y}_i$$

Consider the MSE loss:

$$\ell_{\Theta}(\{\mathbf{x}_i, \mathbf{y}_i\}) = \frac{1}{n} \sum_{i=1}^n \|\mathbf{y}_i - g_{\Theta}(\mathbf{x}_i)\|_2^2$$

Solving for the weights Θ is referred to as **training**.

In general, the loss is **not convex** w.r.t. Θ .

As we have seen, the following **special cases** are convex:

- One layer, no activation, MSE loss ($\Rightarrow$ linear regression).
- One layer, sigmoid activation, logistic loss ($\Rightarrow$ logistic regression).

Training

We train using gradient descent-like algorithms.

Bottleneck: Computation of gradients $\nabla \ell_{\Theta}$.

Training

We train using gradient descent-like algorithms.

Bottleneck: Computation of gradients $\nabla \ell_{\Theta}$.

For the basic MSE, this means:

$$\nabla \ell_{\Theta}(\{\mathbf{x}_i, \mathbf{y}_i\}) = \frac{1}{n} \sum_{i=1}^n \nabla_{\Theta} \|\mathbf{y}_i - g_{\Theta}(\mathbf{x}_i)\|_2^2$$

Training

We train using gradient descent-like algorithms.

Bottleneck: Computation of gradients $\nabla \ell_{\Theta}$.

For the basic MSE, this means:

$$\nabla \ell_{\Theta}(\{\mathbf{x}_i, \mathbf{y}_i\}) = \frac{1}{n} \sum_{i=1}^n \nabla_{\Theta} \|(\mathbf{y}_i - (\sigma(f_{\Theta_n}((\sigma(f_{\Theta_{n-1}}(\cdots(\sigma(f_{\Theta_1}(\mathbf{x}_i)) \cdots))))))\|_2^2$$

Training

We train using gradient descent-like algorithms.

Bottleneck: Computation of gradients $\nabla \ell_{\Theta}$.

For the basic MSE, this means:

$$\nabla \ell_{\Theta}(\{\mathbf{x}_i, \mathbf{y}_i\}) = \frac{1}{n} \sum_{i=1}^n \nabla_{\Theta} \|(\mathbf{y}_i - (\sigma(f_{\Theta_n}((\sigma(f_{\Theta_{n-1}}(\cdots(\sigma(f_{\Theta_1}(\mathbf{x}_i)) \cdots))))))\|_2^2$$

- Computing the gradients **by hand** is infeasible.
- **Finite differences** require $O(\#\text{weights})$ evaluations of ℓ_{Θ} .
- Using the **chain rule** is sub-optimal.

We want to automatize this **computational step** efficiently.

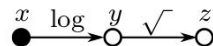
Computational Graphs

Consider a generic function $f : \mathbb{R} \rightarrow \mathbb{R}$.

A **computational graph** is a directed acyclic graph representing the computation of $f(x)$ with **intermediate** variables.

Example:

$$f(x) = \log x + \sqrt{\log x}$$



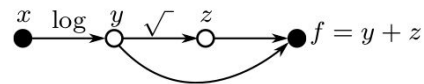
Computational Graphs

Consider a generic function $f : \mathbb{R} \rightarrow \mathbb{R}$.

A **computational graph** is a directed acyclic graph representing the computation of $f(x)$ with **intermediate** variables.

Example:

$$f(x) = \log x + \sqrt{\log x}$$



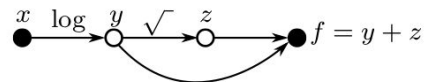
Computational Graphs

Consider a generic function $f : \mathbb{R} \rightarrow \mathbb{R}$.

A **computational graph** is a directed acyclic graph representing the computation of $f(x)$ with **intermediate** variables.

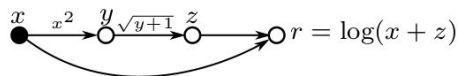
Example:

$$f(x) = \log x + \sqrt{\log x}$$



Example:

$$f(x) = \frac{\log(x + \sqrt{x^2 + 1})}{x^2} - \frac{\log^3(x + \sqrt{x^2 + 1})}{\sqrt{x^2 + 1}}$$



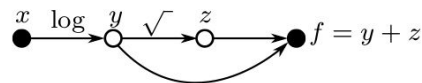
Computational Graphs

Consider a generic function $f : \mathbb{R} \rightarrow \mathbb{R}$.

A **computational graph** is a directed acyclic graph representing the computation of $f(x)$ with **intermediate** variables.

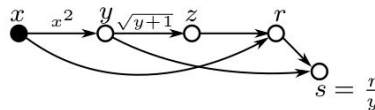
Example:

$$f(x) = \log x + \sqrt{\log x}$$



Example:

$$f(x) = \frac{\log(x + \sqrt{x^2 + 1})}{x^2} - \frac{\log^3(x + \sqrt{x^2 + 1})}{\sqrt{x^2 + 1}}$$



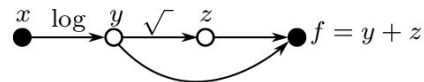
Computational Graphs

Consider a generic function $f : \mathbb{R} \rightarrow \mathbb{R}$.

A **computational graph** is a directed acyclic graph representing the computation of $f(x)$ with **intermediate** variables.

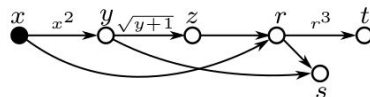
Example:

$$f(x) = \log x + \sqrt{\log x}$$



Example:

$$f(x) = \frac{\log(x + \sqrt{x^2 + 1})}{x^2} - \frac{\log^3(x + \sqrt{x^2 + 1})}{\sqrt{x^2 + 1}}$$



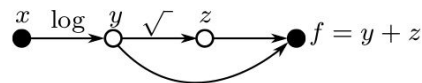
Computational Graphs

Consider a generic function $f : \mathbb{R} \rightarrow \mathbb{R}$.

A **computational graph** is a directed acyclic graph representing the computation of $f(x)$ with **intermediate** variables.

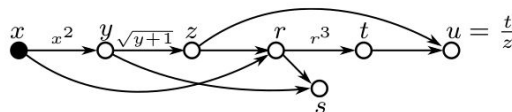
Example:

$$f(x) = \log x + \sqrt{\log x}$$



Example:

$$f(x) = \frac{\log(x + \sqrt{x^2 + 1})}{x^2} - \frac{\log^3(x + \sqrt{x^2 + 1})}{\sqrt{x^2 + 1}}$$



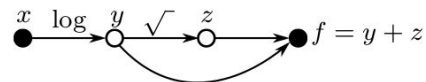
Computational Graphs

Consider a generic function $f : \mathbb{R} \rightarrow \mathbb{R}$.

A **computational graph** is a directed acyclic graph representing the computation of $f(x)$ with **intermediate** variables.

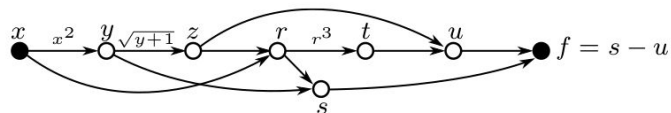
Example:

$$f(x) = \log x + \sqrt{\log x}$$



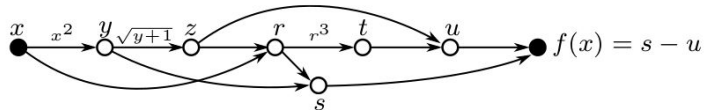
Example:

$$f(x) = \frac{\log(x + \sqrt{x^2 + 1})}{x^2} - \frac{\log^3(x + \sqrt{x^2 + 1})}{\sqrt{x^2 + 1}}$$



Computational Graphs

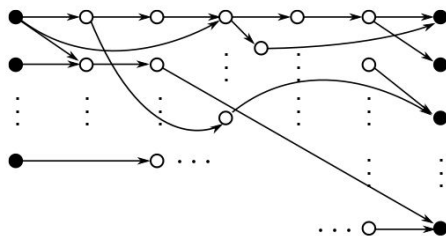
The evaluation of $f(x)$ corresponds to a **forward traversal** of the graph:



The graph is constructed programmatically, for example:

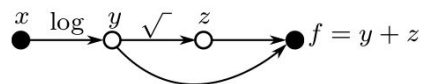
```
z = sqrt(sum(square(x), 1));
```

For **high-dimensional** input/output, the graph may be more complex:



Automatic Differentiation: Forward Mode

$$f(x) = \log x + \sqrt{\log x}$$



$$\frac{\partial x}{\partial x} = 1$$

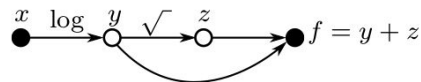
$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial x} \frac{\partial x}{\partial x} = \frac{\partial \log x}{\partial x} \frac{\partial x}{\partial x} = \frac{1}{x} \frac{\partial x}{\partial x}$$

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \frac{\partial y}{\partial x} = \frac{\partial \sqrt{y}}{\partial y} \frac{\partial y}{\partial x} = \frac{1}{2\sqrt{y}} \frac{\partial y}{\partial x}$$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial y} \frac{\partial y}{\partial x} + \frac{\partial f}{\partial z} \frac{\partial z}{\partial x} = \frac{\partial (y + z)}{\partial y} \frac{\partial y}{\partial x} + \frac{\partial (y + z)}{\partial z} \frac{\partial z}{\partial x} = \frac{\partial y}{\partial x} + \frac{\partial z}{\partial x}$$

Automatic Differentiation: Forward Mode

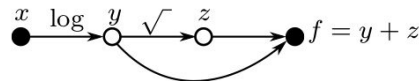
$$f(x) = \log x + \sqrt{\log x} \qquad \frac{\partial f}{\partial x} = \frac{1}{2\sqrt{y}} \frac{1}{x} + \frac{1}{x}$$



Assumption: Each partial derivative is a “primitive” accessible in **closed form** and can be computed on the fly.

Automatic Differentiation: Forward Mode

$$f(x) = \log x + \sqrt{\log x} \qquad \frac{\partial f}{\partial x} = \frac{1}{2\sqrt{y}} \frac{1}{x} + \frac{1}{x}$$



Assumption: Each partial derivative is a “primitive” accessible in **closed form** and can be computed on the fly.

$$\text{cost of computing } \frac{\partial f}{\partial x}(x) = \text{cost of computing } f(x)$$

However, if the input is high-dimensional, i.e. $f: \mathbb{R}^p \rightarrow \mathbb{R}$:

$$\text{cost of computing } \nabla f(\mathbf{x}) = p \times \text{cost of computing } f(\mathbf{x})$$

since partial derivatives must be computed w.r.t. each input dimension.

Automatic Differentiation: Forward Mode

The forward mode computes all the partial derivatives $\frac{\partial y}{\partial x}, \frac{\partial z}{\partial x}, \dots$ with respect to the **input** x .

Straightforward application of the **chain rule**.

Automatic Differentiation: Forward Mode

The forward mode computes all the partial derivatives $\frac{\partial y}{\partial x}, \frac{\partial z}{\partial x}, \dots$ with respect to the **input** x .

Straightforward application of the **chain rule**.

Automatic differentiation $\neq$ Symbolic differentiation
(e.g. autograd) (e.g. Mathematica)

We accumulate values during code execution to generate numerical derivative **evaluations** rather than derivative **expressions**.

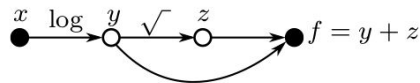
Automatic Differentiation: Forward Mode

The forward mode computes all the partial derivatives $\frac{\partial y}{\partial x}, \frac{\partial z}{\partial x}, \dots$ with respect to the **input** x .

Straightforward application of the **chain rule**.

Automatic differentiation $\neq$ Symbolic differentiation
(e.g. autograd) (e.g. Mathematica)

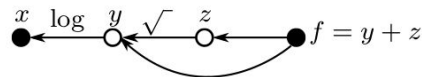
We accumulate values during code execution to generate numerical derivative **evaluations** rather than derivative **expressions**.



Reverse mode: compute all the partial derivatives $\frac{\partial f}{\partial z}, \dots, \frac{\partial f}{\partial x}$ with respect to the **inner nodes**.

Automatic Differentiation: Reverse Mode

$$f(x) = \log x + \sqrt{\log x}$$



$$\frac{\partial f}{\partial f} = 1$$

$$\frac{\partial f}{\partial z} = \frac{\partial f}{\partial f} \frac{\partial f}{\partial z} = \frac{\partial f}{\partial f} \frac{\partial (y + z)}{\partial z} = \frac{\partial f}{\partial f}$$

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial z} \frac{\partial z}{\partial y} + \frac{\partial f}{\partial f} \frac{\partial f}{\partial y} = \frac{\partial f}{\partial z} \frac{\partial \sqrt{y}}{\partial y} + \frac{\partial f}{\partial f} \frac{\partial (y + z)}{\partial y} = \frac{\partial f}{\partial z} \frac{1}{2\sqrt{y}} + \frac{\partial f}{\partial f}$$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial y} \frac{\partial y}{\partial x} = \frac{\partial f}{\partial y} \frac{\partial \log x}{\partial x} = \frac{\partial f}{\partial y} \frac{1}{x}$$

Automatic Differentiation: Reverse Mode

Reverse mode requires computing the values of the **internal nodes** first:

$$\frac{\partial f}{\partial f} = 1$$

$$\frac{\partial f}{\partial z} = \frac{\partial f}{\partial f} \frac{\partial (y+z)}{\partial z} = \frac{\partial f}{\partial f}$$

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial z} \frac{\partial \sqrt{y}}{\partial y} + \frac{\partial f}{\partial f} \frac{\partial (y+z)}{\partial f} = \frac{\partial f}{\partial z} \frac{1}{2\sqrt{y}} + \frac{\partial f}{\partial f}$$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial y} \frac{\partial \log x}{\partial x} = \frac{\partial f}{\partial y} \frac{1}{x}$$

Automatic Differentiation: Reverse Mode

Reverse mode requires computing the values of the **internal nodes** first:

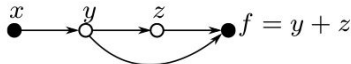
$$\frac{\partial f}{\partial f} = 1$$

$$\frac{\partial f}{\partial z} = \frac{\partial f}{\partial f} \frac{\partial (y+z)}{\partial z} = \frac{\partial f}{\partial f}$$

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial z} \frac{\partial \sqrt{y}}{\partial y} + \frac{\partial f}{\partial f} \frac{\partial (y+z)}{\partial f} = \frac{\partial f}{\partial z} \frac{1}{2\sqrt{y}} + \frac{\partial f}{\partial f}$$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial y} \frac{\partial \log x}{\partial x} = \frac{\partial f}{\partial y} \frac{1}{x}$$

① **Forward pass** to evaluate all the interior nodes $y, z, \dots$



Remark: This is **not** forward-mode autodiff, since we are only computing **function values**.

Automatic Differentiation: Reverse Mode

Reverse mode requires computing the values of the **internal nodes** first:

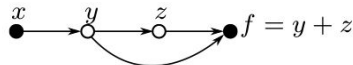
$$\frac{\partial f}{\partial f} = 1$$

$$\frac{\partial f}{\partial z} = \frac{\partial f}{\partial f} \frac{\partial (y+z)}{\partial z} = \frac{\partial f}{\partial f}$$

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial z} \frac{\partial \sqrt{y}}{\partial y} + \frac{\partial f}{\partial f} \frac{\partial (y+z)}{\partial f} = \frac{\partial f}{\partial z} \frac{1}{2\sqrt{y}} + \frac{\partial f}{\partial f}$$

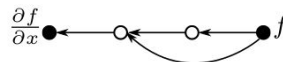
$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial y} \frac{\partial \log x}{\partial x} = \frac{\partial f}{\partial y} \frac{1}{x}$$

- ① **Forward pass** to evaluate all the interior nodes $y, z, \dots$



Remark: This is **not** forward-mode autodiff, since we are only computing **function values**.

- ② **Backward pass** to compute the **derivatives**.



Back-Propagation

When training neural nets, we compute the gradient of a loss

$$\ell : \mathbb{R}^p \rightarrow \mathbb{R}^1$$

where $p \gg 1$ is the number of **weights**.

Back-Propagation

When training neural nets, we compute the gradient of a loss

$$\ell : \mathbb{R}^p \rightarrow \mathbb{R}^1$$

where $p \gg 1$ is the number of **weights**.

Instead of simple derivatives we must compute **gradients** and **Jacobians**.

$$\ell = \epsilon(f_{t-1} \circ f_{t-2} \circ \cdots \circ f_2 \circ f_1)$$

Back-Propagation

When training neural nets, we compute the gradient of a loss

$$\ell : \mathbb{R}^p \rightarrow \mathbb{R}^1$$

where $p \gg 1$ is the number of **weights**.

Instead of simple derivatives we must compute **gradients** and **Jacobians**.

$$\ell = \epsilon(f_{t-1} \circ f_{t-2} \circ \cdots \circ f_2 \circ f_1)$$

Denote by $\mathbf{J}_k$ the Jacobian at layer k .

- **Forward**-mode autodiff:

$$\nabla \ell = \mathbf{J}_{t-1}(\mathbf{J}_{t-2}(\cdots(\mathbf{J}_3(\mathbf{J}_2\mathbf{J}_1)))) \quad \# \text{ ops: } p \sum_{k=2}^{t-1} d_k d_{k+1}$$

Back-Propagation

When training neural nets, we compute the gradient of a loss

$$\ell : \mathbb{R}^p \rightarrow \mathbb{R}^1$$

where $p \gg 1$ is the number of **weights**.

Instead of simple derivatives we must compute **gradients** and **Jacobians**.

$$\ell = \epsilon(f_{t-1} \circ f_{t-2} \circ \cdots \circ f_2 \circ f_1)$$

Denote by $\mathbf{J}_k$ the Jacobian at layer k .

- **Forward**-mode autodiff:

$$\nabla \ell = \mathbf{J}_{t-1}(\mathbf{J}_{t-2}(\cdots(\mathbf{J}_3(\mathbf{J}_2\mathbf{J}_1)))) \quad \# \text{ ops: } p \sum_{k=2}^{t-1} d_k d_{k+1}$$

- **Reverse**-mode autodiff:

$$\nabla \ell = (((\mathbf{J}_{t-1}\mathbf{J}_{t-2})\mathbf{J}_{t-3})\cdots)\mathbf{J}_2)\mathbf{J}_1 \quad \# \text{ ops: } 1 \sum_{k=1}^{t-2} d_k d_{k+1}$$

Back-Propagation

We call **back-propagation** the reverse mode automatic differentiation applied to deep neural networks.

Evaluating $\nabla \ell$ with backprop is as fast as evaluating ℓ .

Back-propagation is not just the chain rule.

In fact, not even the costly forward mode is just the chain rule. There are **intermediate variables**. Backprop is a **computational** technique.

Back-Propagation

We call **back-propagation** the reverse mode automatic differentiation applied to deep neural networks.

Evaluating $\nabla \ell$ with backprop is as fast as evaluating ℓ .

Back-propagation is not just the chain rule.

In fact, not even the costly forward mode is just the chain rule. There are **intermediate variables**. Backprop is a **computational** technique.

Backprop through computational graph of the loss

$\approx$

Backprop “through the network”

Some Observations

- The loss of a MLP will be **non-convex** in general.
 - Multiple **local minima**.
 - Which optimum is reached also depends on the **weight initialization**.
 - In practice, global optimum $\Rightarrow$ **overfitting**.

Some Observations

- The loss of a MLP will be **non-convex** in general.
 - Multiple **local minima**.
 - Which optimum is reached also depends on the **weight initialization**.
 - In practice, global optimum $\Rightarrow$ **overfitting**.
- The loss of a MLP will be **non-differentiable** in general.
 - For example, the ReLU is not differentiable at zero.
 - Software implementations usually return one of the one-sided derivatives.
 - **Numerical issues** are always behind the corner.

Some Observations

- The loss of a MLP will be **non-convex** in general.
 - Multiple **local minima**.
 - Which optimum is reached also depends on the **weight initialization**.
 - In practice, global optimum $\Rightarrow$ **overfitting**.
- The loss of a MLP will be **non-differentiable** in general.
 - For example, the ReLU is not differentiable at zero.
 - Software implementations usually return one of the one-sided derivatives.
 - **Numerical issues** are always behind the corner.
- Effectively training a deep network is far from a solved problem.